

Obsidian Hive — Récapitulatif complet

Anciennement ShieldAI. Renommage "HiveMind" toujours envisagé, pas tranché. Plateforme de cybersécurité autonome pilotée par IA (Alex + Coralie), cible PME africaines francophones en priorité. Projet final du curriculum ML (Phase 9) — **seul projet actif** actuellement.

Statut global : ~76-80% du MVP. Scope final visé : 31 modules. MVP actuel : 5 modules de base + moteur + agents IA + API. **Taille actuelle** : ~192 000 lignes, ~1189 fichiers `.py`. **Deadline visée** : anniversaire de Sam, 21 octobre 2026 — objectif un MVP solide, pas les 31 modules.

1. Les 5 modules de base (terminés)

Module	État	Détail
Scanner	✅ (ML en reprise)	SQLi/XSS/OWASP, ML embarqué (<code>ScannerIA</code>), singleton thread-safe. Fuzzer donne un avis (features) → ML prédit indépendamment → les deux résultats présentés ensemble dans le rapport. Site de test "VulnMart" (Flask, ~23 vulns) construit comme cible. Auth automatique sur site cible via helpers (formulaire, token...).
IDS/IPS	✅	Migré vers Suricata (EVE JSON), scoring temps réel + détection beaconing C2, décision autonome (log/rate-limit/block), audité et profilé (py-spy). Modèle ML entraîné sur place par client (pas de modèle livré fixe) — capture puis fit, protection = code/méthodologie, pas le modèle.
Sandbox	✅ (ML en reprise)	Exécution Docker isolée (13 langages) + honeypot filesystem, <code>BehaviorScorer</code> (patterns MITRE ATT&CK), pipeline ML (AutoEncoder Transformer).
Anti-Phishing	✅	<code>PassiveAnalyzer</code> (18 checks), dataset 2M+ URLs / 36 features, intégration PhishDestroy, TTLCache, suite de tests 44 URLs.
Simulator (AegisRed)	✅	Couverture MITRE ATT&CK complète (Reconnaissance → Persistence), orchestrateur LangGraph, déjà son propre agent intelligent (pas besoin d'un agent central dessus). Branchement en pause en attendant de solidifier son API + Sandbox.

Décision actée : pas de CrewAI nulle part — boucles d'agents manuelles pour garder le contrôle et l'apprentissage (anciens wrappers CrewAI pour Sandbox/Simulator obsolètes, gardés comme trace historique).

2. Les Assets et le moteur

Catégories d'assets

Surveillés dans la durée (via `AssetItem` + `Workflow` + `Queue`) : `WebAsset`, `WebAppAsset`, `NetworkAsset`, `ServerAsset` (voir §3),

`EmailMonitorAsset` (design posé, rien codé)

Actions ponctuelles (appel direct API, pas de moteur) : Email/URL check, Code check

⚠ **Seuls 3 types d'assets réellement utilisables aujourd'hui** : `web_site`, `web_app`, `network`. Server/Email/Code restent des concepts de design ou en cours (Server).

WebAsset / **WebWorkflow**

Scan → **Alex analyse automatiquement** → persistance via `ReportManager`

Code source admin **copié** (jamais utilisé en place) dans `OBSIDIAN_SANDBOX_ROOTS/<asset_id>/`

`fix_allowed` recalculé automatiquement selon présence d'un `source_code_dir` valide

`engine.sync_source_code()` rafraîchit la copie sans changer le chemin

NetworkAsset / **NetworkWorkflow**

3 modes : Gateway, SPAN/Mirroring, Bridge transparent

IDS/IPS en sous-processus isolé (pas un thread), arrêt SIGTERM → attente → SIGKILL, `killpg`

Alex n'est PAS branché sur Network (IDS/IPS a déjà sa propre intelligence temps réel, un LLM serait trop lent/redondant)

Moteur central (ObsidianEngine)

`AssetManager`, `TaskManager`, `WorkflowManager`, `ObsidianManager` (queue `asyncio.PriorityQueue`)

`update_asset()` avec `restart_workflow` conditionnel selon champs modifiés

Config centralisée TOML + validation Pydantic (`ConfigManager`), `.env` pour secrets

`asset_manager.list_by_filter` (status, type, priority, tags) — utilisé par les tools pause/resume de Coralie

3. ServerAsset & agent distant — le gros chantier récent

Pourquoi il existe

NetworkAsset = réseau local où tourne Obsidian Hive

ServerAsset = serveur/réseau ailleurs, nécessite un agent installé sur place

Le protocole agent↔central de ServerAsset débloquera automatiquement le mode distant de NetworkAsset plus tard

Décisions de scope

Option B retenue : agent généraliste multi-capabilities (surveillance + actions à la demande)

Garde-fou non négociable : pas de shell arbitraire — uniquement tools précis/fermés, confirmation humaine obligatoire pour le sensible (même mécanisme que Coralie)

2 façons d'utiliser une capability :

Relayée (logique côté central, agent = tunnel réseau) : Scanner web, Anti-Phishing, Sandbox, Port scan (réutilise celui du Simulateur)

Modules installés localement (lourds) : IDS/IPS, Simulateur — téléchargés à la demande, protégés en binaire compilé (Nuitka/PyArmor envisagés)

Flow d'installation

Admin crée ServerAsset → `install_token` unique/expirant → `curl | bash` → détecte OS, crée user dédié, installe l'agent en service systemd → agent s'enregistre via `/agent/register`, échange le token contre un credential longue durée → canal WebSocket persistant (heartbeat) vers le central.

Sécurité

Secret 32 bytes, transmis une fois, seul le hash gardé côté central (mTLS envisagé plus tard, pas urgent)

Secret/token toujours en header `Authorization: Bearer`, jamais en query string

Pré-hash SHA-256 avant bcrypt (limite 72 octets)

Auto-destruction sans agent root : `uninstall.sh` via règle `sudo` restreinte (`NOPASSWD` scopé)

Fail-closed sur `allowed_tools` : liste vide par défaut

2 niveaux de risque par tool : LOW (pas de confirmation) / MEDIUM-HIGH (confirmation humaine)

Dashboard local

Déployé sur la machine du ServerAsset, authentifié par le credential de l'agent lui-même, scope **strictement limité à cet asset** (jamais de vue sur le central), filtré côté backend par `server_asset_id`.

État du code (validé de bout en bout)

Modèle ServerAsset complet : `install_token`, `AgentStatus`, `AgentCapabilities`, `allowed_tools` fail-closed

Routes REST : `create`, `register` (no-auth), `revoke`, `capabilities add/remove/list`, `tools allow/revoke/list`

WebSocket central `/api/core_ws/ws/server_agent` : auth secret vs hash, heartbeat/ack, réception `tool_result`

Dispatch `manage_server_tool_call` (admin → central → agent, confirmation WSConfirmer)

Cœur de l'agent (`core_agent/`) codé et testé : `config.py` (AgentConfig Pydantic, écriture atomique, permissions 600), `transport.py` (AgentHttpClient httpx+tenacity, AgentWSClient avec heartbeat + reconnexion backoff exponentiel), `dispatcher.py` (routage par `type`), `main.py` (classe Agent, `run_forever()`)

12 tools ServerAsset : `get_system_info`, `check_service_status`, `read_file` (déjà là) + 9 ajoutés (`list_directory`, `disk_usage`, `list_processes`, `search_in_file`, `check_open_ports`, `list_logged_in_users`, `last_logins`, `network_interfaces`, `list_block_devices`)

Scripts bash : `install.sh`, `reregister.sh`, `uninstall.sh` (déclenché à distance par `self_destruct`)

Test réel réussi sur VM Kali : `install` → `register` → `tools` → suppression asset côté central → `self-destruct` → nettoyage complet vérifié (service, dossier, user système)

15 bugs trouvés et corrigés

Condition `_register` inversée · `install_token` requis à tort au restart · secret jamais posé sur `http_client` après register · `await` manquant dans `download_file` · retour incohérent de `_download_tool_engine` · `tool_call` Pydantic passé brut à `json.dumps` · `datetime` non

sérialisable dans `normalize_asset_item` · `generate_secret()` 64 bytes dépassant la limite bcrypt (→ 32 bytes + pré-hash SHA-256) · `asset_id` manquant dans l'URL WS de test · typo `config_updated`/`config_update` · `allowed_tools`/`capabilities` jamais transmis dans `/register` · `KillMode=control-group` empêchait `uninstall.sh` de finir (→ `KillMode=process`) · `StartLimitIntervalSec`/`Burst` mal placés (`[Unit]` pas `[Service]`) · emojis + locale absente plantant le script bash · conflit `libgcc_s` au runtime du binaire Nuitka.

4. Les agents IA

Alex — l'Analyste (✓ validé en conditions réelles, ~95%)

Stateless, conclut toujours par `create_report` (jamais de texte libre sauf conversation anodine sans tool)

10 politiques conditionnelles (prompt injection, secrets, gravité, incertitude, faux positifs...)

Tools : `search_pattern`, `read_file`, `create_file`, `replace_file_content`, `modify_file_content`, `create_report` — confinés, fail-closed

"Vérité mécanique, jamais déclarative" : `_enforce_reliable_diffs`/`_enforce_applied_state` écrasent ce qu'Alex *dit* avoir fait par ce qui s'est *réellement* passé

Validé sur scénario multi-vulnérabilités (SQLi, secrets en dur, MD5 sans sel) avec **Ornith-1.0-9B** — a trouvé les 3 failles, fixé correctement, détecté une prompt injection cachée

Coralie — la Décisionnaire (✓ terminée, ~85%)

Conversationnelle, mémoire persistée (`ConversationManager`), personnalité chaleureuse

Tools auto-découverts (suffixe `_core_tool`), auto-introspection (`get_info_about_tool`)

Catégories : lecture code, consultation rapports/assets, actions non-destructives, actions destructrices (confirmation humaine `@confirm()`),

planification (`JobManager`/`APScheduler`, `SQLAlchemyJobStore`, garde-fou triggers cron vides)

Anti-hallucination : ne jamais réutiliser un `item_id` mémorisé sans le revérifier

"En savoir plus" sur un rapport Alex → ouvre le chat Coralie avec contexte injecté

A récemment reçu un tool pause/resume sur tous les assets à la fois (suite à un test d'attaque simulé où ce manque a été identifié)

5. Infrastructure LLM

`LLMManager` unique et partagé (cohérent avec `models_max=1` de `llama-server`)

Adaptateur multi-provider (`build_request_kwargs`) : OpenAI-compatible vs Anthropic

Streaming via `EventBus` générique, normalisation stream/non-stream identique en aval

`llama-server` (build b9833) avec `--jinja`, presets `models.ini`

Modèles testés : Qwen2.5-3B/7B, Llama-3.1-8B, Qwen3.5-4B, **Ornith-1.0-9B (recommandé, agentic)**

6. Persistence

ReportManager : historique complet par asset, filtres (sévérité/source/date), stats agrégées, compression zstd, **has_fix** en colonne dédiée
ConversationManager : cascade delete, **lazy="selectin"** obligatoire (AsyncSession), **save_agent_turn()** atomique

7. API / Gateway

Auth JWT centralisée (**AuthManager**), routes CRUD assets complètes

POST /agent/alex/analyze (usage manuel), mapping **Source** → **PromptMapping**/**FixId**

handler_wrapper (erreurs de validation → 4xx propres)

/token/refresh (**verify_token_without_exp_verify**)

20/20 tests d'intégration passés

8. Frontend (en cours)

Décision finale : Sam code lui-même, Claude + DeepSeek en supervision ciblée (après échec du 1er essai Google AI Studio, jugé trop générique)

2 modes : automatique et utilisateur-contrôlé

Pas de SaaS pour l'instant, mais architecture pensée pour ne pas bloquer l'ajout futur

Scope V1 complet : assets, chat Coralie, rapports Alex, jobs planifiés, conversations

Config runtime via **config.json** (URL API, nom produit — actuellement "Obsidian Hive")

Style : beau/professionnel/marquand, **pas** terminal/hacker ; logo façon lockup de marque

Couleurs en variables CSS, multi-thèmes dès le départ (pas juste soleil/lune)

Bouton "Analyser avec Alex" sur résultats de modules → route d'analyse WS en direct

UX de chat "à la Claude" (thinking + tool calls temps réel, rejouable depuis les **steps** en DB)

Mode démo (données mockées, zéro requête réseau) + protection des routes en mode non-démo

9. Décisions d'architecture clés

2 agents seulement (Simulator a déjà sa propre intelligence LangGraph)

Reporter = génération déterministe (template), zéro LLM, pour la fiabilité

Confinement fail-closed (sans **OBSIDIAN_SANDBOX_ROOTS**, aucun accès fichier)

Agent distant toujours en connexion sortante (jamais entrante)

Vérité mécanique > déclarative partout où c'est possible

Actions destructrices = confirmation humaine obligatoire, jamais d'exécution directe

10. Bugs mémorables (hors ServerAsset, pour ne pas les refaire)

`.scalars()` au lieu de `.all()` → résultats corrompus silencieusement

`await` manquants sur coroutines (`engine.start()`, `_get_report_manager()`) → composant jamais démarré, aucune erreur visible

`os.path.exists` (typo) au lieu de `exists`

`get_running_loop()` dans un thread sans event loop → nécessite `run_coroutine_threadsafe`

`pause_all_jobs`/`resume_all_jobs` n'existent pas dans APScheduler (seulement `pause_job`/`resume_job`)

Trigger cron sans champ temporel → se déclenche toutes les secondes

11. TrustSignal (module Deepfake d'Obsidian Hive)

Détection de contenu généré par IA (texte + images)

Architecture : XLM-RoBERTa contrastive encoder, handcrafted features, StackingML, fusion transformer custom

Marché cible : Afrique francophone/Maghreb

3 Go de textes labellisés déjà réunis, majoritairement en anglais

Bloqué par la puissance machine — plan : entraînement sur Colab (upload Drive → Colab pour les gros volumes)

12. Prisma (assistant multi-agent)

CrewAI/LangGraph, FastAPI, React

Produit standalone **ET** interface conversationnelle d'Obsidian Hive

Vision aussi comme concurrent Claude/ChatGPT pour le marché africain